PanoptOS: Early Bot Detection from Sparse Behavioral Snapshots in Old School RuneScape

Author Withheld

Abstract

We present PanoptOS, a methodology for detecting automated accounts ("bots") in the MMORPG Old School RuneScape. When Jagex bans an account it disappears from the public Hiscores, yielding a free but delayed binary label. We cast Hiscore table selection during active discovery as a nonstationary multi-armed bandit, with arms sampled by probability matching on the product of a per arm ban rate and a per arm discovery rate. Discovered accounts feed into a dual branch sequence model: a causal transformer encoder with continuous time Fourier encoding over 173 behavioral features, supervised at every observation prefix and gated with a character level transformer over the account's display name to capture the shared naming conventions used by automated farms. We evaluate via a *time horizon* protocol that reports detection quality as a function of post discovery observation length, capturing how quickly the model becomes useful rather than only its asymptotic ceiling. The model achieves ROC-AUC above 0.98 within four days of discovery, reaching 0.9865 at a 30-day post discovery horizon.

1 Introduction

Old School RuneScape (OSRS) is a massively multiplayer online role-playing game operated by video game developer and publisher Jagex Limited, in which players develop a single persistent character by accumulating experience (XP) across 25 skills and completing activities such as boss encounters and minigames. Skill progression is deliberately slow and repetitive, and rare drops and tradeable resources can be exchanged in-game for *coins*, the game's primary currency, which third-party black markets in turn convert to real-world money; this gives automated play a direct monetary incentive and makes large scale botting an endemic problem that Jagex actively bans against. Jagex maintains a public per account ranking service, the *Hiscores*, that exposes, through an unauthenticated HTTP endpoint, each ranked account's cumulative XP in every skill and the per account count for roughly 85 ranked activities, refreshed shortly after the account logs out. Although designed as a competitive leaderboard, the Hiscores have become the de facto public telemetry surface for the third-party OSRS ecosystem and are the sole input consumed by the work presented here.

Detecting automated agents from behavioral traces is a recurring problem across online platforms, including click fraud, review manipulation, and game economy exploitation. OSRS is an unusually clean instance. When an account is banned, it drops from the Hiscores, and an external observer with no privileged access can therefore build a labeled training set of behavioral trajectories for millions of accounts, with binary labels that materialize as accounts disappear.

What such an observer does not see is the rich first-party signal that an internal detector at Jagex is likely to weight most heavily, such as payment token and chargeback history, IP and ASN co-occurrence across accounts, browser and device fingerprints, the per session unique browser identifier (UBID), and fine-grained clickstream and client side patterns (input timing, mouse and keyboard dynamics, client telemetry), among others. Jagex is also likely to apply graph based

methods over in game trade and interaction networks to surface real-world trading (RWT) rings. The relational structure of gold flows is exactly the kind of signal that is invisible from a per account public leaderboard. Jagex may also batch enforcement into periodic ban waves rather than acting on every detection immediately, which deliberately starves bot operators of the per account feedback they would otherwise use to localize what the detector keys on. The resulting detection surface is opaque enough that even the banned population cannot reliably reverse engineer it, and botters routinely develop cargo cult theories about why a particular account was lost, attributing outcomes to superstitions about play patterns, client configurations, or proxy choices that Jagex would never confirm and may not in fact use.

Within MMORPGs generally, prior bot detection work has analyzed server side movement patterns in World of Warcraft [1], party-play log structure in Lineage [2], and gold farming trade networks [3]. These academic approaches all assume operator side data access. The closest external observer precedent is a community run effort rather than a peer reviewed system, which we discuss in §2.

Three difficulties dominate. First, observable accounts are nonuniformly informative. Bot incidence varies by orders of magnitude across Hiscore tables, so uniform sampling wastes most of the budget. Second, labels arrive late (days to weeks after bot activity begins) and are themselves the output of Jagex's own classifier with unknown recall, so the negative class is contaminated. Third, operational value requires early detection, since a classifier that needs 30 days of history is of little use if Jagex typically bans within a few weeks.

This paper presents PanoptOS, with two contributions:

1. A bandit formulation of active discovery that selects arms by *probability matching over Bayesian utility estimates*, combining a Laplace smoothed per arm ban rate over time decayed observations with a discount smoothed per arm discovery rate to handle nonstationarity and arm saturation.
2. A dual branch sequence model: a causal transformer encoder [5] with a continuous time Fourier embedding [6] over behavioral features and a character level transformer over the account name, gated by the behavioral representation, supervised at every observation prefix, and calibrated via length conditioned Platt scaling on a positive capped validation set.

We also evaluate along a post discovery time horizon axis rather than at a single operating point, since the operationally relevant question is how quickly the model becomes useful, not its asymptotic ceiling.

2 Community Run Bot Detection

A separate, community led effort predates the present work and already operates from the "external observer" position this paper adopts. That effort is the open source *Bot Detector* plugin [4] for the third-party OSRS client RuneLite. The plugin runs client side on participating players' machines and reports nearby account observations (display name, world number, in-game location, equipped gear, and Grand Exchange gear value) to a centralized server. The deployed classifier is a tree based model that scores each candidate from a single Hiscores snapshot. Although richer client side signals (gear, location, world) are collected, they are not consumed by the production model. Labels are obtained the same way as in this work (via subsequent disappearance of an account from the Hiscores), so the supervisory signal is identical, and neither system requires cooperation from Jagex. Bot Detector is, to our knowledge, the only prior ML based bot detection effort in OSRS that does not assume access to Jagex-internal data.

PanoptOS differs from Bot Detector primarily along the temporal axis. The plugin's deployed classifier reduces an account to a single Hiscores snapshot and emits a point classification, with no explicit notion of when the snapshot was taken or how the account's stats got there. PanoptOS instead models the full post discovery sequence of XP and activity count snapshots, with the

elapsed time between samples encoded directly into the input (§7); the contributions in §7 and §9 are about extracting signal from how an account's trajectory evolves over weeks rather than from where it sits at a single instant. A secondary difference is in how candidates are discovered. Bot Detector observes accounts which the plugin's user population walks past, while PanoptOS pulls the Hiscores API directly under a learned crawl policy (§4). The contributions of this work can be read, in part, as asking how far the same supervisory signal can be pushed when the input is a longitudinal sequence rather than a single snapshot. The supervisory signal exploited by both systems (subsequent disappearance from the Hiscores) was first operationalized at scale in OSRS by Bot Detector, and the open source release of their methodology is what makes a direct comparison on identical labels possible here. We benchmark directly against a reproduction of the Bot Detector methodology in §10.1.

3 Problem Setting and Label Generation

Let $\mathcal{A}$ denote the space of OSRS accounts observable via the public API. For each account $a \in \mathcal{A}$, we observe a sparse time series $(s_a(t_1), s_a(t_2), \ldots)$ where $s_a(t) \in \mathbb{R}^d$ is a 110-dimensional vector of cumulative experience points across 25 skills and counts across approximately 85 activities. Timestamps t_i correspond to queries initiated by the crawler, not to in-game events, so the sampling rate is determined by the requery policy. Consecutive observations with an identical state vector are deduplicated: within each maximal run of an unchanged state we retain only its first and last observation, so a dormant account polled repeatedly contributes the onset and the most recent observation of its frozen state rather than one snapshot per query. The retained timestamps are thus a subset of the crawler's queries; the elapsed time between them is preserved and supplied to the model, so an unchanged span is represented as a single wide interval rather than a run of zero change steps. All snapshot counts reported here are post deduplication.

A label $y_a \in \{0, 1\}$ is observed when a transitions to a terminal state. Three terminal states are distinguished using the RuneMetrics profile API: NOT_A_MEMBER (banned, $y_a = 1$), NO_PROFILE (deleted, unlabeled), and PROFILE_PRIVATE (privacy hidden, unlabeled). Accounts that remain in the Hiscores after a minimum observation period are treated as $y_a = 0$.

This label generation process introduces two structural biases. *Label delay*: Jagex's enforcement pipeline is itself a classifier with unknown latency, so $y_a = 1$ is observed only after Jagex's system has fired. *Negative contamination*: bots that are never detected by Jagex are absorbed into the negative class. This is the standard Positive-Unlabeled (PU) regime [7]. We treat unlabeled as negative and rely on the small estimated contamination rate to keep classifier risk tractable; Platt calibration on a positive capped validation set (§8.3) provides an empirical correction to the probability scale. A more principled treatment would adopt nnPU style risk estimators [7] or a logistic head with a class-prior correction, which we discuss as a natural extension in §11.

Table 1: Dataset summary. Counts reflect the post filter dataset: accounts with discovery age ≥ 60 days, at least three snapshots, and total experience $\geq$ 100k XP. Splits are deterministic by account ID hash.

Accounts	1,802,893
of which banned	358,721 (19.9%)
Snapshots	58,789,815
Mean snapshots per account	32.6
Account name length (median; p_{25}, p_{75})	10 (8, 11)
Train / Validation split	90% / 10%

4 Active Discovery as a Multi-Armed Bandit

The OSRS Hiscores are organized as K paginated tables across three hiscore variants (main, level-3 skillers, 1-Defence pures). Each table sorts accounts by a specific metric (a skill XP total, an activity count, or overall XP) and is capped at 80,000 pages of 25 rows, for an upper bound of 2,000,000 ranked names per table. The tables are heavily overlapping. A single account that plays the game broadly will appear simultaneously on the overall XP table, on every skill table whose XP it has accumulated, and on every activity table whose count it has recorded, so the union of tables covers far fewer than $K \times 2{,}000{,}000$ distinct accounts. Bot density also varies sharply across tables, and gathering skill tables see order of magnitude more bots than boss kill tables. A discovery policy that samples uniformly would spend most of its rate limited budget on low yield arms. We therefore formulate table selection as a K-armed bandit.

Two further details matter for how PanoptOS uses the tables. First, the level-3 skillers and 1-Defence pures variants are *sticky*: an account that has ever qualified remains listed on those tables even after subsequent training breaks the qualifying condition, so the variant tables accumulate accounts whose current state may no longer match the variant's nominal definition. Second, the paginated tables are used only for name discovery; once an account name has been observed on any table, all snapshots used for modeling are pulled from the general per account Hiscores endpoint, which returns the most recent XP and activity counts across every skill and activity simultaneously. The sorted rank that originally surfaced the account is not used as a feature.

This structure imposes a coverage constraint on the population the crawler can ever see. An account is reachable only if it appears on at least one Hiscore table, and the probability that the bandit eventually finds a given reachable account grows with the number of tables it occupies. An account that ranks on overall XP plus several skill and activity tables can be discovered through any of those arms, while one that only just clears a single table's cutoff has exactly one route in. Accounts whose totals never cross any table's cutoff are by construction invisible to PanoptOS regardless of how the bandit allocates its budget.

4.1 Ban Rate Estimate

Each table k has an unknown ban probability θ_k: the probability that an account *discovered on that table* is subsequently banned. We estimate it with a Laplace smoothed ratio over time decayed observations,

$$\hat{\theta}_k \;=\; \frac{1 + W_k^1}{2 + W_k^1 + W_k^0},$$

where, summing over the set $\mathcal{D}_k$ of accounts ever discovered on table k with age a_i (days since discovery) and label $y_i \in \{0, 1\}$ (1 banned, 0 not banned),

$$W_k^y \;=\; \sum_{i \in \mathcal{D}_k \,:\, y_i = y,\; a_i \geq \delta} \exp\!\left(-\frac{a_i - \delta}{\tau}\right), \quad y \in \{0, 1\}.$$

Observations with $a_i < \delta$ are excluded entirely so that bans have time to materialize before they enter the estimate. The remaining contributions are damped on a timescale τ, downweighting stale observations smoothly rather than at a hard upper cutoff. In our deployment $\delta = 7$ days (minimum lookback) and $\tau = 30$ days (decay timescale). The constants 1 and 2 in the numerator and denominator are the standard Laplace smoothing offsets, equivalent to a uniform Beta prior under a binomial likelihood; they keep $\hat{\theta}_k$ well-defined for arms with few or zero observations and pull it toward $1/2$ when evidence is thin.

The ban rate estimate is recomputed periodically from the discovery database rather than maintained incrementally, so $\hat{\theta}_k$ tracks the current decay weighted history rather than an unbounded accumulation. Stochasticity is introduced at the selection step (§4.3) rather than within $\hat{\theta}_k$ itself, so the estimate is deterministic conditional on the observed data.

4.2 Discovery Rate and Arm Saturation

Ban rate alone is insufficient. An arm with high historical ban rate can be fully saturated, so every page returns accounts already in the database, and no new positives are discovered regardless of θ_k. We therefore introduce a second per arm quantity, the *discovery rate* $\rho_k \in [0, 1]$, defined as the probability that a fetch from arm k yields a newly seen active account.

We track ρ_k with a discount smoothed running estimate. Per arm counts (a_k, b_k) of new and not new outcomes are updated each fetch by

$$a_k \leftarrow \gamma\, a_k + \mathbf{1}[\text{new active}], \qquad b_k \leftarrow \gamma\, b_k + \mathbf{1}[\text{not new active}],$$

with discount factor $\gamma \in (0, 1)$. The geometric forgetting gives a fixed point effective sample size of $1/(1-\gamma)$ observations, so the estimate tracks recent behavior rather than accumulating indefinitely; in our deployment $\gamma = 0.996$, yielding an effective window of 250 observations. The point estimate is

$$\hat{\rho}_k = \frac{a_k}{a_k + b_k}.$$

Equivalently, (a_k, b_k) parameterize a discount decayed Beta(a_k, b_k) posterior over ρ_k, with $\hat{\rho}_k$ its posterior mean. Smoothing the numerator and denominator on the same timescale, rather than smoothing the rate directly, lets the prior pseudocounts anchor the estimate before observations arrive and decay smoothly as they do, mirroring the role of the Laplace offsets in $\hat{\theta}_k$.

We place a uniform prior over ρ_k, initializing every arm at $a_k = b_k = 1$ irrespective of table size. The initial point estimate is therefore $\hat{\rho}_k = 1/2$ for all arms at an effective sample size of two, matching the Beta$(1, 1)$ offset used for $\hat{\theta}_k$. Observations overwhelm the prior within a few hundred fetches.

4.3 Probability Matching over Bayesian Utility Estimates

The per arm utility estimate is the product

$$\hat{u}_k = \hat{\theta}_k \cdot \hat{\rho}_k,$$

which is proportional to the expected number of *eventually banned newly discovered* accounts per fetch under the smoothed estimates of the two factors. The factorization makes the tradeoff explicit, since an arm earns budget only when it both contains bots and has capacity to yield new accounts. The two factors are also nonstationary in operationally distinct ways. Ban waves move θ_k in discrete steps as bot populations are suppressed, while ρ_k drifts smoothly as a table fills with already discovered accounts. Keeping them as separate factors lets each be smoothed on the timescale appropriate to its own dynamics, rather than averaged into a single estimator that would have to compromise between them.

We sample arms by *probability matching* on $\hat{u}_k$, mixed with a uniform exploration floor:

$$p_k = (1 - \varepsilon)\, \frac{\hat{u}_k}{\sum_{j=1}^{K} \hat{u}_j} + \frac{\varepsilon}{K},$$

with $\varepsilon = 0.02$ in our deployment. The first term assigns budget proportional to estimated utility; the second guarantees a nonzero pull rate for every arm regardless of its current $\hat{u}_k$, so an arm whose statistics have collapsed early on can recover as new observations arrive.

The standard critique of probability matching as suboptimal relative to greedy argmax is a stationary regime result and does not apply here. Under the drift just described, hedging budget in proportion to current utility is the conservative choice; argmax would overcommit to whichever arm happened to lead at the moment of a regime change.

Exposing exploration as a single scalar ε rather than as randomness drawn from the smoothing machinery keeps the rate independent of each arm's current effective sample size. Effective

sample sizes vary widely across arms, since the ban rate estimate sees decay weighted history while the discovery rate estimate sees a $1/(1-\gamma)$-step window, and a fixed exploration rate is easier to tune against the fetch budget than one that drifts as those quantities evolve.

Within a selected table, the page index is sampled from a $\mathrm{Beta}(2,1)$ distribution scaled to the table's current effective page range. The resulting density is triangular on $[0,1]$ with mean $2/3$, biasing sampling toward lower ranked (higher page index) accounts. Lower ranked accounts have less cumulative XP and are typically earlier in their lifecycle, so discovering them deep in a table maximizes the post discovery observation window for both classes. We catch the account near the start of its trajectory rather than after most of its formative behavior has already happened before discovery.

5 Adaptive Requery Scheduling

Known accounts must be requeried to detect bans, but a uniform cadence wastes budget on dormant accounts and reacts slowly when active ones accumulate new behavior between queries. We use an adaptive schedule that is continuous in account age and dormancy.

Both account age and dormancy enter through the same one sided saturation kernel,

$$s(t;h) \;=\; 1 - 2^{-t/h},$$

which rises from 0 at $t=0$ to 1 as $t \to \infty$, passing through $\frac{1}{2}$ at one half-life h.

Let g be days since discovery. The base requery interval is set by account maturity,

$$\beta(g) \;=\; \beta_{\text{fresh}} + (\beta_{\text{mature}} - \beta_{\text{fresh}})\, s(g;h_g),$$

interpolating from β_{fresh} for a freshly discovered account (polled hardest) up to β_{mature} for a mature one. The rationale is a bot survival prior. The only available notion of account age is tenure in our own database, a *left-truncated* survival signal: bots are banned and drop out of the active population, so persistence without disappearing is evidence against an account being a short-lived bot, regardless of when it was first observed. The ban hazard is concentrated early, so most of the discriminating evidence accrues within the first weeks, and the base interval should saturate to a fixed ceiling rather than grow without bound. We set the half-life h_g to the median observed time to ban and use $(\beta_{\text{fresh}}, \beta_{\text{mature}}, h_g) = (0.5, 1.0, 16.5)$ days; a fresh account, being more likely a bot, is thus polled twice as often as a mature one at the same dormancy.

Let d be days since the account's observed state last changed. A dormancy multiplier stretches the base interval for quiet accounts,

$$\mu(d) \;=\; 1 + (\mu_{\max} - 1)\, s(d;h_d),$$

from 1 for an active account up to $\mu_{\max}$ for a long dormant one, reaching the midpoint after one dormancy half-life h_d. Anchoring at $d=0$ is deliberate: since $s(0;h_d)=0$, an active account incurs no dormancy penalty and is polled at exactly its base interval. A one sided kernel is the right choice here, as dormancy lives on the half-line $d \geq 0$; a transition centered away from the boundary would never return exactly 1 at the active end. The transition is also smooth, avoiding the thundering herd behavior of a piecewise schedule when many accounts cross a threshold at once. We use $\mu_{\max} = 7$ and half-life $h_d = 14$ days.

The final interval is perturbed by multiplicative uniform jitter of $\pm 20\%$ to desynchronize the queue:

$$\tau \;=\; \beta(g)\,\mu(d)\,\big(1 + U(-0.2, 0.2)\big).$$

Dormancy d is the time since the account entered its current observable state, not since the last successful query. An account queried daily for 30 days whose state has not changed is dormant under this definition, and the schedule treats it as such.

6 Feature Construction

For each snapshot we construct a 173-dimensional feature vector in four groups: raw ($d = 110$), velocity ($d = 25$), ratio ($d = 24$), and derived ($d = 14$) behavioral statistics. Feature construction is deterministic and shared between training and inference.

6.1 Raw, Velocity, and Ratio Features

Raw features are $\log(1 + x)$ of each skill's cumulative XP and each activity's count. The log compresses the dynamic range (0 to 2×10^8 for XP) and stabilizes optimization. Velocities are per skill XP gain rates between consecutive snapshots, $v_{i,k} = (x_{i,k} - x_{i-1,k})/\Delta t_i$. Ratios express per skill share of total XP, $r_{i,k} = x_{i,k}/x_{i,\text{overall}}$. This normalization removes the overall level confound and makes behavioral profiles comparable across account strengths.

6.2 Derived Features

The derived features target statistical signatures that distinguish scripted from human play, in three clusters.

6.2.1 Concentration and entropy

Let $p_{i,k}$ denote the normalized share of skill or activity k at snapshot i. We compute: normalized skill entropy $H_i^{\text{skill}} = -\sum_k p_{i,k} \log p_{i,k} / \log K_{\text{skill}}$, a corresponding activity entropy over kill count based activities (excluding rank and score based activities whose scales and defaults differ), and max share ratios $m_i^{\text{skill}} = \max_k p_{i,k}$ for skills and m_i^{act} analogously for activities. Low cumulative entropy indicates a narrow training profile, characteristic of gathering bots.

6.2.2 Gain distribution signals

Cumulative entropy is a weak signal in isolation. A long-lived human account that once had a narrow profile retains that narrowness in XP totals even after diversifying. We therefore compute *interval level* analogues on per snapshot gain vectors. *Gain entropy* is the normalized entropy of the per skill XP gain vector in the current interval; bots produce near zero gain entropy (all gains in one skill), while humans spread gains across multiple skills even in a single session. *Active skills* and *active activities* count how many skills and activities changed in each interval, $A_i^{\text{skill}} = |\{k : x_{i,k} > x_{i-1,k}\}|$ (and analogously for activities), which discriminates strongly. A bot grinding a single skill has exactly one active skill almost every interval. Activity gains here and below are computed over kill count activities only: rank and score based activities move in both directions, so upward rating fluctuations would otherwise register as spurious gains. *Gain cosine similarity* is the cosine similarity between the per skill gain vectors of consecutive intervals; a scripted loop reproduces the same gain profile every interval (similarity near 1.0), while human variation produces much lower values.

6.2.3 Persistence and variability

Activity switch is a binary flag indicating whether the single dominant activity (by KC gain) changed between consecutive intervals; *activity streak* counts the number of consecutive intervals with the same single dominant activity. Humans switch; bots grinding a single boss produce long streaks. *Velocity delta* is the change in total XP per hour between consecutive intervals, compressed with a signed logarithm $\text{sign}(\delta) \log(1 + |\delta|)$. Bots hold a steady rate (deltas near zero); humans are bursty, swinging in both directions.

6.3 Sequence Construction

Sequence length is itself a leaky feature. Banned accounts vanish from the dataset shortly after enforcement, while surviving accounts continue to accumulate snapshots, so positives have systematically shorter trajectories than negatives, not for behavioral reasons but because Jagex caught them. A model trained on fully observed trajectories will pick this up and use length itself as a ban indicator, a heuristic that generalizes nowhere. At deployment, prefix length is set by how long the account has been observed, not by whether it is eventually banned.

Per prefix supervision breaks this dependency. Each training example is a window of up to $T_{\max} = 30$ consecutive snapshots; for trajectories longer than $T_{\max}$, the window placement is drawn uniformly, providing mild temporal augmentation. Within each window, interior snapshots are additionally thinned at a per sequence rate drawn uniformly from $[0, 0.25)$, with the window's first and last snapshots always retained. The motivation parallels the length argument: the collector's revisit cadence is itself label correlated, since the scheduler of §5 adapts each account's polling interval to its current state, so a model shown only contiguous windows could learn to read the collector's own schedule. Thinning varies the observed cadence so that predictions rest on the trajectory rather than on how often we happened to sample it. Under the causal mask of §7, the model emits a prediction at every position t conditioned only on snapshots $1, \ldots, t$, and the loss supervises all of them (§8). Every prefix length therefore appears as a training example for both classes: the early snapshots of every long-lived legitimate account contribute short prefix negatives at exactly the lengths where banned accounts are overrepresented, and no prediction can condition on where the stored sequence happens to end.

At evaluation time, sequences are truncated by *prefix time* rather than by count. Given a horizon Δt, only snapshots with $t \leq \text{discovered\_at} + \Delta t$ are retained. This is the fundamental operation underlying the time horizon evaluation of §9.

7 Model Architecture

The detection model has two branches: a feature branch over the 173-dim behavioral sequence, and a name branch over the account's display name. The branches are combined via a learned gate and fed into a three-layer MLP classifier at every sequence position. Figure 1 shows the full architecture; the remainder of this section describes each component. §10.1 motivates the choice of a sequence model over a tree based baseline empirically.

7.1 Feature Branch: Time Aware Transformer Encoder

Operationally, the behavioral transformer treats each Hiscores snapshot as one token. For an account with up to $T_{\max} = 30$ retained snapshots, each token contains the 173 behavioral features computed at that query time. The model first standardizes those features, projects them into a 96-dimensional hidden space, and adds a learned projection of the snapshot's elapsed time since discovery. The transformer then lets every snapshot attend to every *earlier* snapshot under a causal mask, producing at each position a contextualized representation of the account's history up to that observation. Each per position representation is concatenated with the gated name representation and passed to the classifier head, yielding one logit per observed prefix; at deployment the account is scored by the final position's logit, conditioned on the full retained window.

Formally, $x \in \mathbb{R}^{T \times d_x}$ with $d_x = 173$ and $T \leq T_{\max} = 30$ denotes a behavioral feature sequence of true length $L_x \leq T$ (we drop the batch index for clarity). Each row x_t carries an associated elapsed time $\tau_t \in \mathbb{R}_{\geq 0}$, in days since the account's discovery, recorded at query time. Inputs are channel wise z-scored against training set statistics (denoted S) and lifted from d_x to $C = 96$ model dimensions by a per step linear projection W^x.

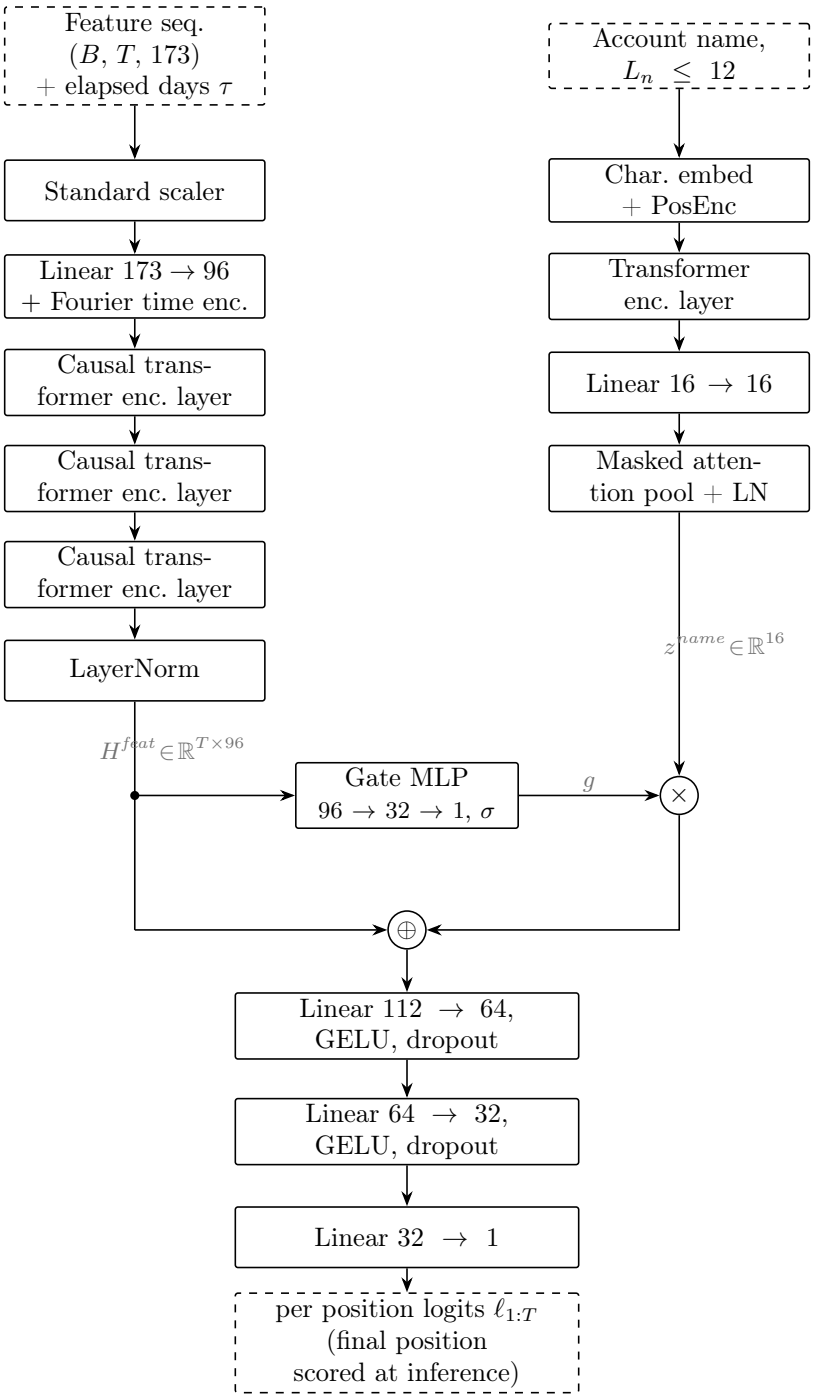

Figure 1: **PanoptOS architecture.** Two branches process the behavioral feature sequence (left) and the account display name (right) in parallel. The feature branch projects the 173-dim input to $C = 96$ channels, adds a sinusoidal Fourier embedding of elapsed days τ, and applies $L = 3$ pre-norm causal transformer encoder layers with $H = 4$ heads each, producing per position states $H^{\text{feat}} \in \mathbb{R}^{T\times 96}$ in which position t sees only snapshots $1, \ldots, t$; the name branch embeds characters at $d_{\text{model}} = 16$, applies a single transformer encoder layer with two heads, and pools to $z^{\text{name}} \in \mathbb{R}^{16}$. At each position a scalar gate $g_t = \sigma(\text{MLP}(H_t^{\text{feat}})) \in (0, 1)$ scales the name representation; the (ungated) per position feature state and the gated name representation are concatenated into $\mathbb{R}^{112}$ and passed through a three-layer MLP head, yielding one logit per observed prefix. Standard input scaling is folded into the deployed model; at inference the final unpadded position's logit is mapped to a probability by length conditioned Platt scaling (§8.3). B: batch size; $T \leq 30$: feature sequence length; $L_n \leq 12$: name length.

Continuous time Fourier encoding. The crawler's requery schedule (§5) yields highly nonuniform sampling, so encoding raw position indices would discard the actual time elapsed between snapshots. We therefore map elapsed time directly through a sinusoidal Fourier embedding in the spirit of the original transformer encoding [5] but applied to a continuous input [6]: with $F = 16$ log spaced periods $P_f \in [0.4, 365]$ days and angular frequencies $\omega_f = 2\pi/P_f$, define

$$\psi(\tau) \;=\; \big[\sin(\omega_1\tau),\,\ldots,\,\sin(\omega_F\tau);\;\cos(\omega_1\tau),\,\ldots,\,\cos(\omega_F\tau)\big] \;\in\; \mathbb{R}^{2F}.$$

A linear map $W^\tau : \mathbb{R}^{2F} \to \mathbb{R}^C$ projects this into the model dimension and is added to the projected feature input:

$$h_t^{(0)} \;=\; W^x x_t \;+\; W^\tau \psi(\tau_t), \qquad t = 1, \ldots, T.$$

Log spacing the periods spans behaviorally meaningful timescales (daily, weekly, monthly, annual) without committing to any single one.

Encoder stack. The trunk is $L = 3$ pre-norm transformer encoder layers with $H = 4$ heads each, feedforward width $4C = 384$, GELU activation, and dropout $p = 0.2$. Attention is causal: position t attends only to positions $t' \le t$, so each output state is a function of the observation prefix alone. Padding is handled by a key padding mask $m_t^{\text{feat}} = \mathbf{1}[t > L_x]$ that excludes padded positions from attention. Letting $\mathcal{T}_\ell$ denote the ℓ-th encoder layer, the trunk computes

$$H^{\text{feat}} \;=\; \big(\mathcal{T}_{L-1} \circ \cdots \circ \mathcal{T}_0\big)\big(h^{(0)},\, m^{\text{feat}}\big) \;\in\; \mathbb{R}^{T\times C},$$

with $h^{(0)}$ the time augmented projected input above and the input scaler S folded into $W^x x$.

Choice of sequence backbone. Two natural alternatives were considered for this branch before settling on the transformer encoder. The first was a causal dilated TCN [8], attractive for its exponentially growing receptive field and full training time parallelism. The second was an LSTM [9], the standard recurrent choice for modeling sequences of behavioral snapshots. Both were viable. In internal experiments, asymptotic ($\Delta t = 30$) metrics were broadly comparable across the three backbones, but the transformer pulled ahead on the early horizon summary (§10). itAUC_{PR} over $\Delta t \in \{4, \ldots, 30\}$ was higher by roughly five PR-AUC percentage points than the best convolutional configuration, with the bulk of the improvement concentrated in the $\Delta t \le 7$ days regime. Two factors appear to drive the gap. First, attention gives every snapshot direct access to every earlier snapshot, where the TCN routes information through a fixed dilation geometry and the LSTM through a strictly sequential state. At the short prefixes that matter for early detection, the handful of available snapshots can condition on one another directly. Second, replacing the implicit step index ordering with an explicit Fourier embedding of elapsed time makes the highly nonuniform sampling cadence visible to the model. A TCN treats consecutive positions as equally spaced regardless of the underlying $\tau_t - \tau_{t-1}$ gap, and an LSTM has no native mechanism for conditioning its gates on real elapsed time.

7.2 Attention Pooling

The name branch aggregates its variable length character sequence by a learned attention pool. For an encoder output $h \in \mathbb{R}^{T\times C}$ with true length $L \le T$, a small MLP $a : \mathbb{R}^C \to \mathbb{R}$ ($C \to C/2 \to C/4 \to 1$ with GELU) produces per position scores $e_t = a(h_t)$, which are length masked ($\tilde{e}_t = -\infty$ for $t > L$, else $\tilde{e}_t = e_t$), softmaxed, and aggregated:

$$\alpha_t \;=\; \frac{\exp(\tilde{e}_t)}{\sum_{t'=1}^{T} \exp(\tilde{e}_{t'})}, \qquad \text{Pool}_C(h, L) \;=\; \sum_{t=1}^{T} \alpha_t\, h_t.$$

The $-\infty$ mask matters. Without it the softmax normalizes over padded positions, bleeding zero valued contributions into every example's representation and breaking length invariance. The operator is instantiated at $C_n = 16$ on the name branch; the feature branch is left unpooled, since the per position classifier head (§7.4) consumes its full output.

7.3 Name Branch: Character Level Transformer

Account names carry real signal, since bot farms often share naming conventions (sequential digits, random alphanumerics, shared prefixes). The branch is intentionally shallow. The information budget is tiny (at most $T_n = 12$ characters), and extra capacity risks overfitting to spurious name patterns that correlate with ban incidence in the training distribution but do not generalize. Tokenization is at the character level rather than via a subword scheme such as BPE or WordPiece. With names capped at twelve characters, subword vocabularies would collapse to single characters on most names and carve already short strings into one or two tokens on the rest, hiding the positional structure (sequential digits, shared prefixes) the branch exists to detect. Combined with the gate of §7.4, the shallow architecture treats the name signal as a corroborating cue rather than a primary feature.

Formally, $n \in \{0, 1, \ldots, V\}^{T_n}$ with $V = 65$, padding symbol 0, and $T_n = 12$ denotes the integer encoded display name of true length $L_n \le T_n$. With embedding matrix $E \in \mathbb{R}^{(V+1)\times d}$ ($d = 16$, row $E_{0,:}$ fixed at zero by the padding index) and a learned positional encoding $P \in \mathbb{R}^{T_n \times d}$, the per position embedding is

$$u_t \;=\; E_{n_t,:} \;+\; P_{t,:}, \qquad t = 1, \ldots, T_n.$$

A single transformer encoder layer [5] $\mathcal{T}$ with two heads, feedforward width $2d$, dropout, and key padding mask $m_t = \mathbf{1}[t > L_n]$ produces $\mathcal{T}(u, m) \in \mathbb{R}^{T_n \times d}$. A linear projection $W^{\text{name}} \in \mathbb{R}^{d \times C_n}$ ($C_n = 16$), the masked attention pool of the previous subsection, and a final LayerNorm yield

$$z^{\text{name}} \;=\; \mathrm{LN}\big(\mathrm{Pool}_{C_n}(\mathcal{T}(u, m)\, W^{\text{name}},\; L_n)\big) \;\in\; \mathbb{R}^{C_n}.$$

7.4 Gated Combination

A scalar gate, computed at every position from the behavioral representation by a shallow MLP $\mathrm{MLP}_g : \mathbb{R}^C \to \mathbb{R}$ ($C \to 32 \to 1$ with GELU), modulates the name representation before concatenation:

$$g_t \;=\; \sigma\big(\mathrm{MLP}_g(H_t^{\text{feat}})\big) \;\in\; (0, 1), \qquad z_t^{\text{comb}} \;=\; \big[\, H_t^{\text{feat}} \,;\; g_t \cdot z^{\text{name}} \,\big] \;\in\; \mathbb{R}^{C + C_n},$$

with the pooled name representation broadcast across positions. The construction targets a classic multimodal failure mode in which a purely additive combination would let the weaker name signal pull predictions away from a confident behavioral branch. Conditioning the gate on H_t^{feat} implements a simple form of confidence aware fusion, so the name contributes when the behavior is ambiguous and is suppressed when it is not. The learned gate does not collapse to a constant. The classifier head maps each z_t^{comb} to a per position logit ℓ_t; at inference, the final unpadded position's logit is mapped to a probability by the length conditioned Platt scaling of §8.3.

8 Training

8.1 Dataset Construction and Splitting

Three filters are applied: discovery age ≥ 60 days so bans have time to materialize, at least three snapshots, and total experience $\ge$ 100k XP. We use a 90%/10% train/validation split with no held out test set; the validation set serves both early stopping and final reporting. Splits are a

deterministic function of the account identifier, not of time, so that a given account's trajectory is entirely in one split. A temporal split would break label completeness for the validation set, since accounts banned after the cutoff would be mislabeled as negatives. Identity splits preserve completeness at the cost of overlapping time ranges across splits, which is the right tradeoff here.

8.2 Optimizer and Loss

We optimize per position binary cross-entropy with logits using AdamW and a cosine learning rate schedule over the training horizon. The sequence label is broadcast to every position with at least $L_{\min} = 3$ observed snapshots (inference never scores shorter prefixes); per position losses are averaged within each sequence before averaging over the batch, so long trajectories do not dominate the gradient. No class weighting is applied; the post filter positive rate is 20% (Table 1), and early experiments with class weighted BCE did not improve PR-AUC.

Checkpoint selection targets the deployment objective directly: after each epoch we run the single pass time horizon sweep of §9 on the validation set, scored with that epoch's calibration (§8.3), and retain the epoch maximizing itAUC_{PR}. Full window PR-AUC is dominated by mature, fully observed accounts and is nearly insensitive to gains in the early horizon regime the system exists to serve.

8.3 Probability Calibration

Neural classifiers trained with BCE on imbalanced, PU contaminated data typically produce well ranked but poorly calibrated probabilities. For operational deployment, calibration matters independently of ranking quality. We apply a length conditioned extension of Platt scaling [10]:

$$P(y = 1 \mid s, n) \;=\; \sigma(as + c \log n + b),$$

where s is the model logit and n the number of snapshots in the scored window, fit on validation logits after each epoch. The length covariate exists because per prefix scoring evaluates accounts at widely varying maturities and the logit distribution drifts with snapshot count; conditioning on $\log n$ absorbs that drift, so a single decision threshold carries the same expected precision at every maturity. The covariate is centered during the fit for conditioning, with the offset folded back into b.

Two details matter. First, the validation set is *positive capped* to a target rate of 10% before the Platt fit. The current dataset rate is approximately 20% (Table 1), but this rate inflates over time as banned accounts accumulate; 10% is our estimate of the long run operational base rate. Without the cap, Platt fits a large negative intercept and drags calibrated scores toward zero throughout. Second, calibration is folded into the deployed model so inference returns probabilities directly; the parameters (a, b, c) are stored with the model and applied as part of the forward pass, with n recovered from the padding mask.

9 Time Horizon Evaluation

Standard binary classification metrics (ROC-AUC, PR-AUC) tell an incomplete story in this setting. A classifier that is excellent given 30 days of post discovery data but uninformative given 3 days is operationally unhelpful. By day 30 Jagex has typically already banned the account, and the external classifier's value has evaporated. We therefore evaluate across a time horizon axis.

9.1 Protocol

The per position head makes the sweep a single forward pass. Each validation account is windowed to its first $\Delta t_{\max} = 30$ days of post discovery history, and the causal model emits one

logit per snapshot position. For each horizon Δt in the grid $\mathcal{H} = \{4, 5, \ldots, 30\}$ days, the account is scored by its logit at the last position within Δt of discovery, calibrated with that position's snapshot count (§8.3); this is exactly the score the deployed model would have emitted at that point in the account's life. Positions with fewer than $L_{\min} = 3$ snapshots are never scored, so an account enters the evaluation cohort at the first horizon by which it has three snapshots and remains for all later horizons. The cohort is therefore fixed across horizons up to data availability, and movement along the curve reflects the model rather than population churn. At each horizon we record ROC-AUC, PR-AUC, and sample count. The grid starts at $\Delta t = 4$ rather than $\Delta t = 3$ because the only accounts scoreable within three days are those producing three snapshots in three days, an atypical high frequency subpopulation whose composition is not representative of the short prefix regime we want to characterize.

9.2 Integrated Time-AUC

Early detection quality is summarized by the trapezoidal integral of the AUC curve along the time axis, normalized to the horizon range:

$$\text{itAUC} \;=\; \frac{1}{\Delta t_{\max} - \Delta t_{\min}} \int_{\Delta t_{\min}}^{\Delta t_{\max}} \text{AUC}(\Delta t)\, d\Delta t.$$

Two models with identical asymptotic AUC can have substantially different itAUC. A classifier that reaches AUC 0.9 by day 3 and plateaus outscores one that reaches 0.95 only at day 21, because the former is usable during the window where Jagex's enforcement latency permits external value. A single operating point protocol conflates the asymptotic ceiling with the rate of approach to it; itAUC separates them, and the per horizon curve shows the tradeoff. itAUC thus answers the question: how quickly does this model become useful?

9.3 Additional Diagnostics

We complement the AUC curves with several diagnostics: horizon stratified DET curves, plotted log-log to surface behavior at very low false positive rates; a reliability diagram with expected calibration error; the empirical CDF of calibrated scores by class; and the distribution of model scores across horizons (median, IQR, and p_{10}–p_{90} bands), with banned and normal accounts overlaid on a single panel. A well-behaved model produces a clear rightward shift in the banned score distribution as the horizon grows while the normal score distribution remains pinned near zero, so that the class supports stay disjoint at every horizon.

10 Empirical Results

We evaluate the trained model on a held out account disjoint validation set using the diagnostic suite defined in §9. Figure 2 reports the full panel.

Retained sequence discrimination. As a no horizon sanity check, we also score each validation account on its latest retained sequence, capped at $T_{\max} = 30$ snapshots rather than truncated at a fixed calendar horizon. Under this protocol the calibrated model achieves ROC-AUC = 0.9948 (95% CI $[0.9945, 0.9951]$) and PR-AUC = 0.9849 (95% CI $[0.9841, 0.9856]$) on 169,733 scoreable accounts (Table 3). This is not the $\Delta t = 30$ point on the time horizon curve: the 30-day calendar horizon keeps only snapshots with $t \leq$ discovered_at + 30 days, scores 136,600 accounts, and reaches ROC-AUC = 0.9865 and PR-AUC = 0.9631 (Table 5). The retained sequence result confirms that the model has learned a strong ranking signal when later history is available, but it is not the operational figure of merit.

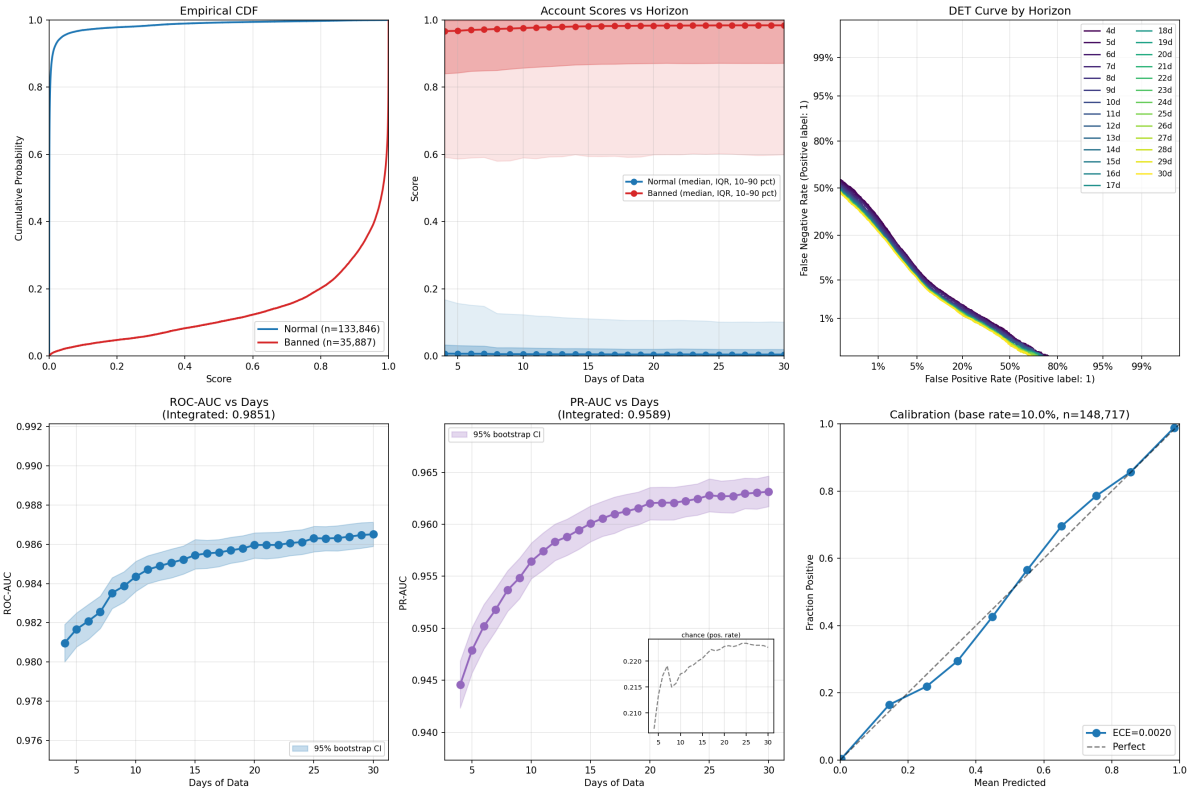

Figure 2: **Full evaluation suite.** *Top row*: empirical CDF of calibrated model scores by class; percentile bands (p_{10}–p_{90}, IQR, median) of model scores as a function of post discovery horizon, with banned (red) and normal (blue) accounts overlaid on a single panel; DET curves stratified by horizon, plotted log-log to surface behavior at very low false positive rates. *Bottom row*: ROC-AUC and PR-AUC as functions of post discovery horizon on a linear scale (with integrated time-AUC summaries $\text{itAUC}_{\text{ROC}}$ and itAUC_{PR} inset, and the per horizon positive rate, the chance level PR-AUC, shown in a subpanel of the PR plot); reliability diagram with expected calibration error at the 10% operational base rate assumption.

Time horizon behavior. The bottom left and bottom middle panels plot the headline metrics against post discovery observation length on a linear scale. The integrated summaries are $\text{itAUC}_{\text{ROC}} = 0.9851$ and $\text{itAUC}_{\text{PR}} = 0.9589$ over $\mathcal{H} = \{4, \ldots, 30\}$ days. The ROC-AUC curve is already above 0.98 at $\Delta t = 4$ and reaches 0.9865 by $\Delta t = 30$, so the early detection regime targeted in §9 is operationally accessible. The horizon stratified DET curves (top right) show the same gain at low FPR.

Score trajectories by class. The top middle panel shows the early detection mechanism directly. The median banned account score (red) shifts rightward and the banned percentile bands tighten as the horizon grows, with the banned p_{10}–p_{90} band rising well above 0.5 within the first several days; the normal account bands (blue) stay pinned near zero across the full horizon range, and the normal p_{90} remains far below the lower edge of the banned p_{10}–p_{90} band. The overlay shows that modest observation is enough for actionable scores within the window Jagex's enforcement latency permits, without eroding the class separation that keeps the false positive rate low.

Calibration. The reliability diagram (bottom right) shows that length conditioned Platt scaling on the positive capped validation set yields a near diagonal predicted versus empirical curve, with expected calibration error of 0.0020 measured on the same positive capped 10% validation set used for the Platt fit. Calibrated probabilities are therefore directly interpretable as per account ban likelihoods rather than relative rankings, which matters when downstream consumers combine PanoptOS scores with other signals.

10.1 Comparison to the Bot Detector Baseline

The closest prior ML based bot detection system in OSRS without operator side data is the Bot Detector plugin [4][1] (§2), whose deployed classifier scores each candidate from a single Hiscores snapshot using a tree based model. We reproduce that methodology as our baseline on identical labels and splits. The reproduction is a `DecisionTreeClassifier` (Gini criterion, max depth 30, minimum samples per leaf 20, grid searched over the deployed system's hyperparameter grid) trained on a single snapshot per account, with input the 110 raw skill XP and activity count features of the most recent snapshot, matching the deployed system's inputs. At horizon Δt the baseline scores the most recent snapshot within the prefix; the transformer consumes the full snapshot sequence within the same prefix.

As Figure 3 shows, PanoptOS improves upon Bot Detector's current methodology at every horizon on both metrics, with the gap concentrated in the short horizon regime that matters operationally. The integrated summaries are $\Delta\,\text{itAUC}_{\text{ROC}} = 0.035$ (0.9851 vs. 0.9500) and $\Delta\,\text{itAUC}_{\text{PR}} = 0.064$ (0.9589 vs. 0.8949). At $\Delta t = 4$ days the PR-AUC gap is roughly eight percentage points; by $\Delta t = 30$ days it has narrowed to under six as the baseline's most recent snapshot becomes more informative on its own. The narrowing is the expected shape, since a snapshot taken late in an account's observable life carries most of what a single instant can carry, and the trajectory closes the remaining gap. The PR-AUC gap is the operationally relevant one, because PR-AUC weights the high precision regime, which is exactly where short horizon evaluation pushes the model.

The residual signal beyond a single snapshot is order dependent. XP gain and activity trajectories carry information in the sequence of values, not just in the most recent one. A single snapshot classifier can split on "current Jad kill count" or "cumulative XP in skill k"; unlike a sequence model, it cannot condition on whether high velocity preceded or followed

[1] We thank the Bot Detector project for the open release of their methodology and model code, and for answering questions about their deployed system, without which this direct comparison on identical labels would not be possible.

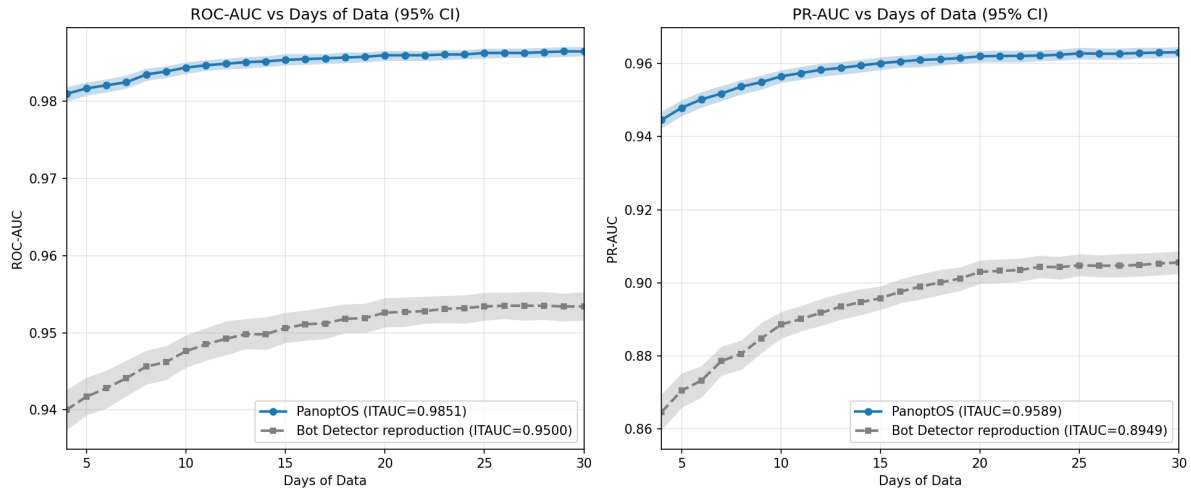

Figure 3: **PanoptOS vs. Bot Detector reproduction.** ROC-AUC (left) and PR-AUC (right) as a function of post discovery horizon $\Delta t \in \{4, \ldots, 30\}$ days. The baseline is a single decision tree on the most recent snapshot within each horizon, reproducing the deployed Bot Detector methodology [4]; the transformer consumes the full snapshot sequence within the horizon. Integrated time-AUC summaries appear in the legend. The shaded band visualizes 95% stratified bootstrap confidence intervals.

a switch in dominant activity, on the correlation between consecutive gain vectors, or on the position of a change-point within the observed window.

10.2 Post Training Int8 Quantization

The deployment target is a Raspberry Pi 5, so reducing memory footprint and arithmetic cost on a CPU without a dedicated matrix unit is a hard constraint. We therefore apply PyTorch dynamic int8 quantization to the trained float model. Weights of all eligible linear layers (the feature branch transformer blocks, the name branch character transformer, the gating MLP, and the classifier head) are statically quantized to int8, while activations are quantized on the fly per forward pass. No calibration set is required, and the Platt scaling layer fit in §8.3 is reused unchanged on top of the quantized backbone. The quantized model is exported to ONNX and served via ONNX Runtime at inference time. Because dynamic quantization derives activation scales from each batch at run time, scores depend mildly on batch composition, so the inference worker sorts each scoring window by sequence length before batching, keeping batches nearly padding free and homogeneous.

Table 2 reports the impact on the held out validation set. Full sequence ROC-AUC moves by 7×10^{-5} and PR-AUC by 2.2×10^{-4}, both nominally in the int8 model's favor, which at this magnitude is sign noise rather than a real improvement. Operating points move by at most 7×10^{-4} in either direction, and calibration drifts mildly (ECE -0.9×10^{-3}, Brier -0.3×10^{-3}). Per account score agreement remains tight (Pearson 0.9986, mean absolute difference 0.0048), consistent with quantization acting as small noise on the score rather than a systematic shift.

The horizon conditioned view tells the same story: ROC-AUC and PR-AUC at $\Delta t \in \{4, 7, 14, 21, 30\}$ days are each within $\sim 8 \times 10^{-4}$ and $\sim 2.6 \times 10^{-3}$ of the float reference, respectively, with the gap largest at the shortest horizon and narrowing as the horizon grows. Dynamic int8 quantization therefore costs little at this scale, and the deployment grade model is the int8 backbone plus the original length conditioned Platt scaler.

Table 2: Effect of post training dynamic int8 quantization on the held out validation set. *Score drift* statistics summarize per account differences between float and quantized scores. ECE in this table is measured at the validation set's accumulated base rate rather than the 10% operational prior used elsewhere; only the float–int8 delta is of interest here, and both columns share the same population.

Metric	Float	Int8	Δ
ROC–AUC	0.99481	0.99488	+0.00007
PR–AUC	0.98486	0.98509	+0.00022
ECE	0.01731	0.01642	−0.00090
Brier score	0.02113	0.02079	−0.00034
Precision @ recall 0.90	0.96898	0.96878	−0.00020
Precision @ recall 0.95	0.92473	0.92513	+0.00040
Recall @ precision 0.90	0.96506	0.96575	+0.00070
Recall @ precision 0.95	0.92981	0.92964	−0.00017
Predicted positive rate	0.19598	0.19694	+0.00096
Score drift			
Mean $\lvert\Delta s\rvert$		0.00476	
$p_{95}\,\lvert\Delta s\rvert$		0.02652	
$p_{99}\,\lvert\Delta s\rvert$		0.08496	
Pearson correlation		0.99865	
Per horizon AUC deltas (int8 − float)			
$\Delta t = 4$ days	$\Delta\text{ROC} = -0.00082$, $\Delta\text{PR} = -0.00256$		
$\Delta t = 7$ days	$\Delta\text{ROC} = -0.00063$, $\Delta\text{PR} = -0.00183$		
$\Delta t = 14$ days	$\Delta\text{ROC} = -0.00040$, $\Delta\text{PR} = -0.00113$		
$\Delta t = 21$ days	$\Delta\text{ROC} = -0.00037$, $\Delta\text{PR} = -0.00095$		
$\Delta t = 30$ days	$\Delta\text{ROC} = -0.00034$, $\Delta\text{PR} = -0.00093$		

11 Discussion

11.1 New Accounts vs. Account Takeovers

The positive class as labeled blends two structurally different bot origins. *Newly created bots* are bot controlled from account creation; the publicly visible trajectory is entirely bot behavior, and both branches of the model see a coherent signal in which velocity and ratio fingerprints align with the label and the display name was generated by the same operator that runs the bot. *Account takeovers*, where a legitimate account is compromised (credential stuffing, RATs, phishing, or real-world trade purchase) and switched to bot operation mid-life, present a different problem. From the Hiscores' perspective the trajectory contains a change-point separating legitimate from automated behavior, and the human chosen display name is by construction uninformative.

The architectural choices in §7 cut both ways for takeovers. Continuous time positional encoding and self-attention over the full sequence let the model, in principle, locate the change-point and weight the post takeover prefix more heavily; a sudden drift in velocity or skill ratio is exactly the signal it is trained to use. The gated name branch, however, is positively misleading on these accounts, since the display name predates the bot and looks human. The gate is intended to suppress the name signal when behavioral evidence is informative, which mitigates the issue without eliminating it. The current dataset does not distinguish the two origin types, so we cannot quantify the difference in detection quality. Treating takeover detection as explicit change-point inference on the trajectory, and pairing it with a discovery policy that selects from established accounts rather than from the bandit's high yield bot tables (§4), is a natural extension we leave to future work.

11.2 Limitations

PU learning. Treating unlabeled accounts as negatives is a convenience, not a principled choice. Under PU assumptions, the BCE risk estimator underestimates the true classification risk [7]. An nnPU objective, or a logistic head with a class-prior correction using Jagex's known enforcement rate as prior, would likely improve quality in the high precision regime, where negative contamination hurts most.

Distributional drift. Jagex's enforcement is adversarial and drifting; bot populations coevolve with it. The current methodology retrains periodically and relies on the rolling bandit window to keep discovery statistics current, but does not monitor or correct for drift in the feature distribution or the ban rate estimates directly. Calibration drift on held out recent data, paired with change-point detection on the feature streams, would address both gaps.

Coverage. The methodology can only see bots that survive long enough to appear on the Hiscores. Jagex bans many bot accounts before they accumulate the XP needed to occupy a leaderboard, and those accounts are by construction invisible to a Hiscores only observer. The bots PanoptOS targets are therefore the longer-lived ones, precisely the population that has already evaded Jagex's faster detection paths and that human players are most likely to encounter in-game.

Dependence on Jagex's enforcement latency. The methodology assumes bot lifetimes are long enough to accumulate several snapshots before enforcement. Under perfect enforcement, positives would vanish from the Hiscores before a snapshot was recorded, and the labeling mechanism would collapse. Jagex's latency is in practice substantial and variable, and the classifier occupies that gap. Faster or slower enforcement regimes would require recalibrating the discovery and requery windows.

12 Conclusion

We presented a methodology for bot detection in Old School RuneScape, comprising bandit driven active discovery with a decomposed utility, a continuous requery scheduler, a dual branch sequence model with a gated name signal and a positive capped Platt fit, and time horizon evaluation summarized by itAUC. Each component targets a specific structural feature of the problem; together they keep the early detection regime central to evaluation rather than reduced to a single number.

References

[1] Mitterhofer, S., Krügel, C., Kirda, E., and Platzer, C. (2009). Server-Side Bot Detection in Massively Multiplayer Online Games. *IEEE Security & Privacy*, 7(3), 29–36.

[2] Kang, A. R., Woo, J., Park, J., and Kim, H. K. (2013). Online Game Bot Detection Based on Party-Play Log Analysis. *Computers & Mathematics with Applications*, 65, 1384–1395.

[3] Kwon, H., Mohaisen, A., Woo, J., Kim, Y., Lee, E., and Kim, H. K. (2017). Crime Scene Reconstruction: Online Gold Farming Network Analysis. *IEEE Transactions on Information Forensics and Security*, 12(3), 544–556.

[4] Bot Detector Project. *Bot Detector: A RuneLite Plugin for Crowdsourced Bot Detection in Old School RuneScape*. Open-source software, 2021–present. https://github.com/Bot-detector/bot-detector.

[5] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention Is All You Need. In *Advances in Neural Information Processing Systems 30*.

[6] Tancik, M., Srinivasan, P. P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J. T., and Ng, R. (2020). Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains. In *Advances in Neural Information Processing Systems 33*.

[7] Kiryo, R., Niu, G., du Plessis, M. C., and Sugiyama, M. (2017). Positive-Unlabeled Learning with Non-Negative Risk Estimator. In *Advances in Neural Information Processing Systems 30*.

[8] Bai, S., Kolter, J. Z., and Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. *arXiv:1803.01271*.

[9] Hochreiter, S., and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780.

[10] Platt, J. (1999). Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In *Advances in Large Margin Classifiers*.

A Per Feature Empirical CDFs

For each scalar feature used by the model, we plot the empirical cumulative distribution function over the training set, separated by class label (*Normal* in blue, *Banned* in red). Larger horizontal separation between the two CDFs corresponds to greater per feature discriminative power; near coincident curves carry little class information and contribute primarily through interactions in the transformer encoder. Plots are organized by feature group, mirroring the construction in §6.

A.1 Raw skill XP

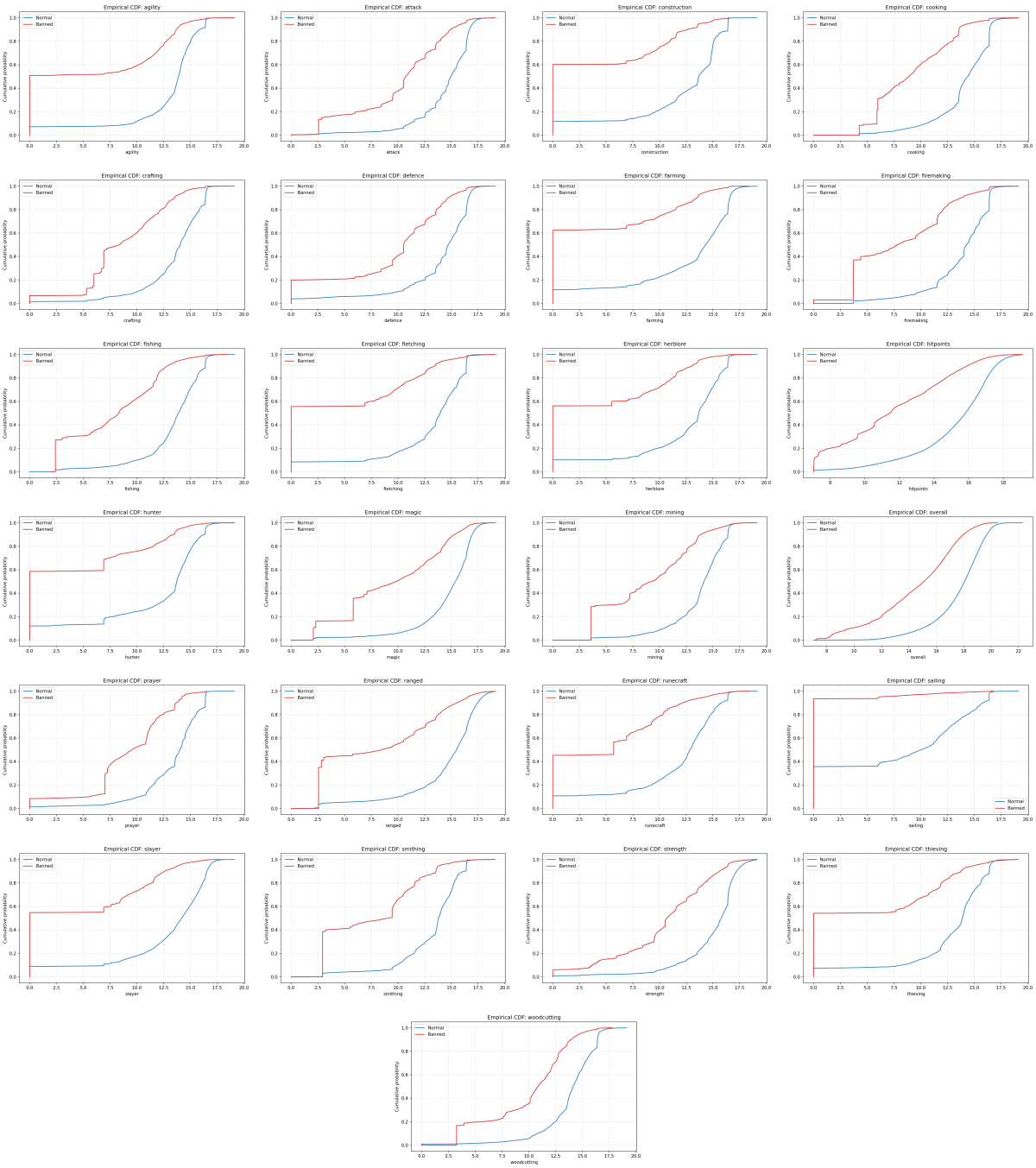

Figure 4: Empirical CDFs of $\log(1+x)$-transformed cumulative skill XP across the 25 OSRS skills, by class.

A.2 Raw activity counts

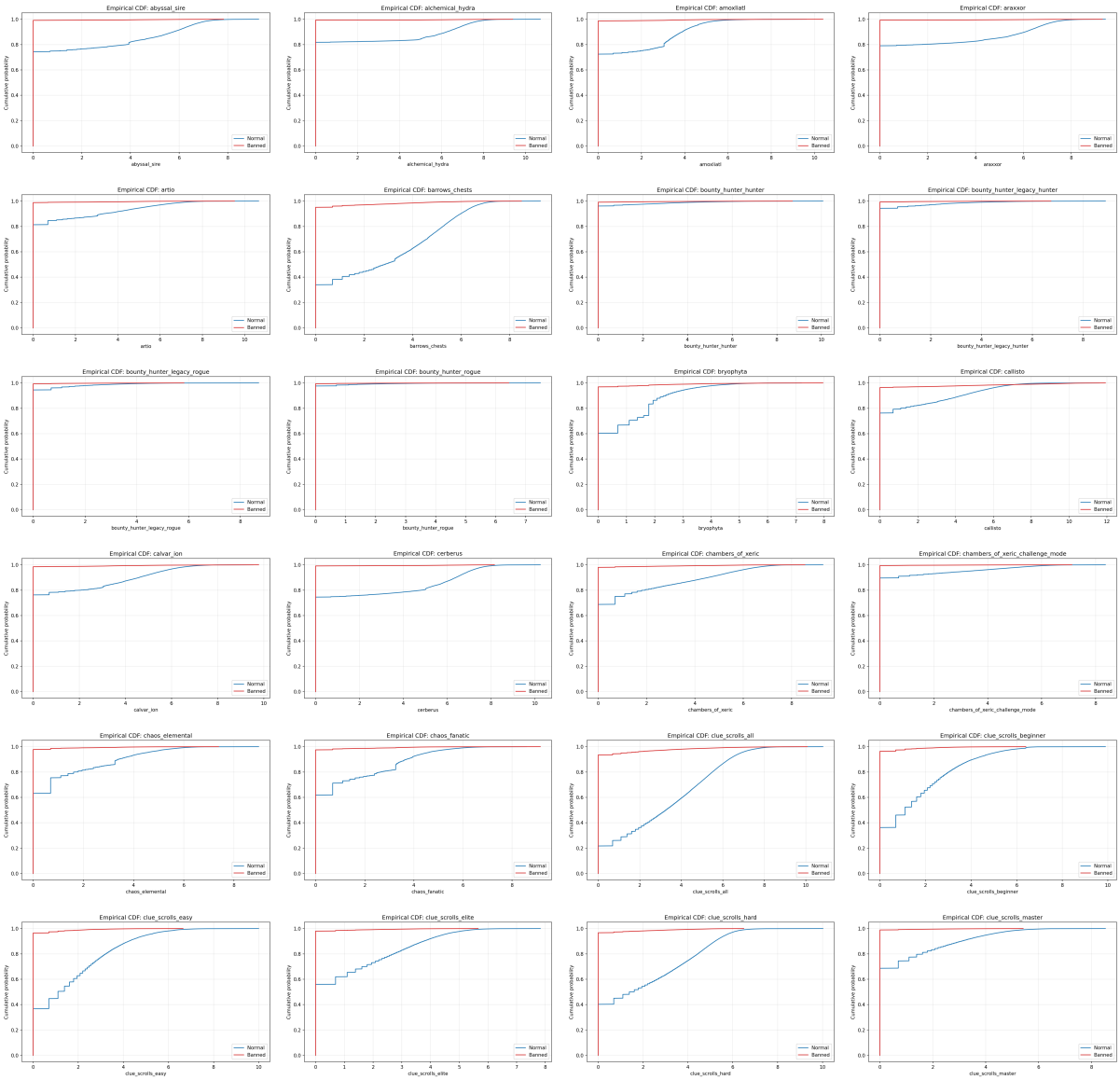

Figure 5: Empirical CDFs of raw activity counts, alphabetical (1/4).

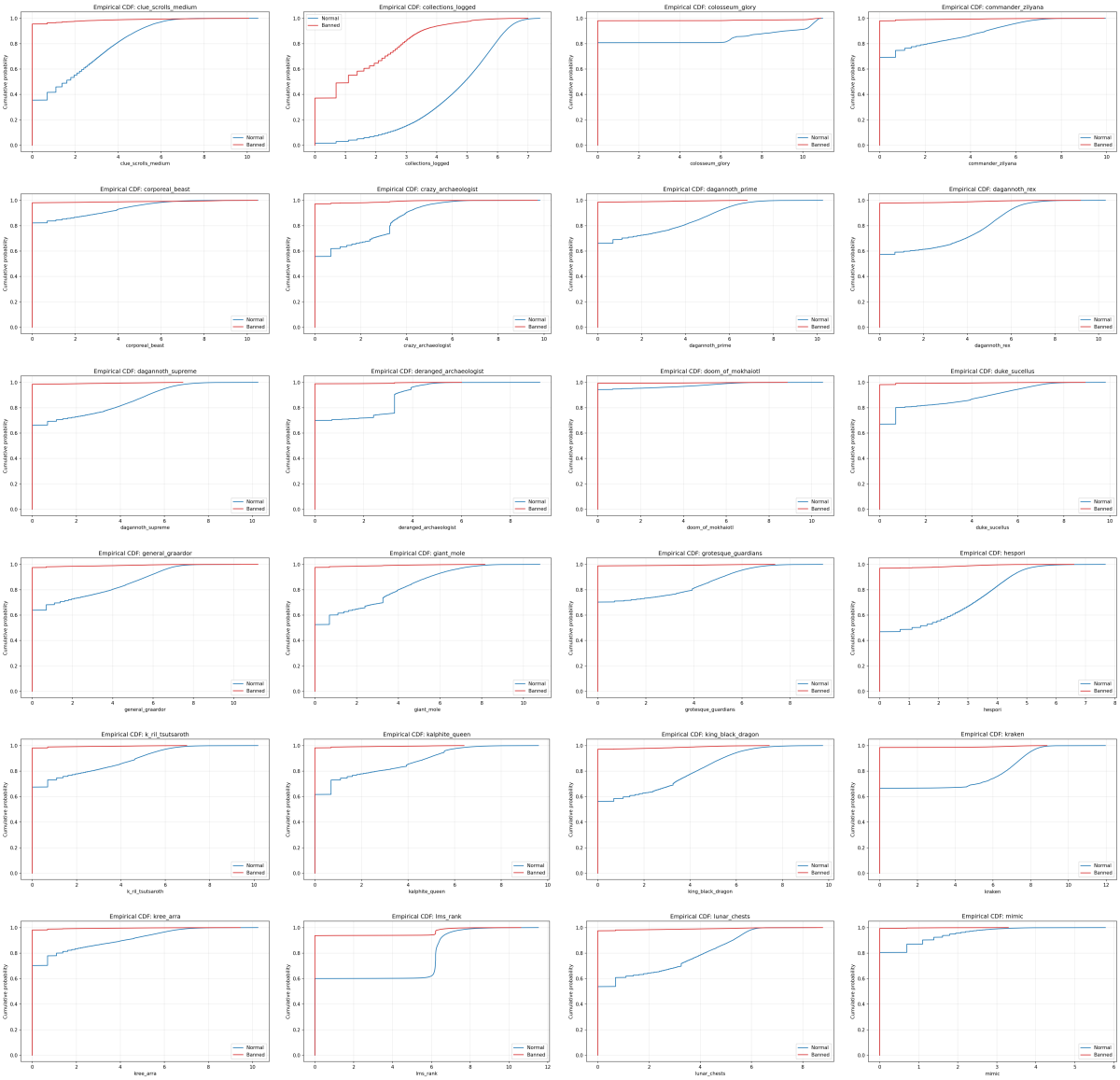

Figure 6: Empirical CDFs of raw activity counts, alphabetical (2/4).

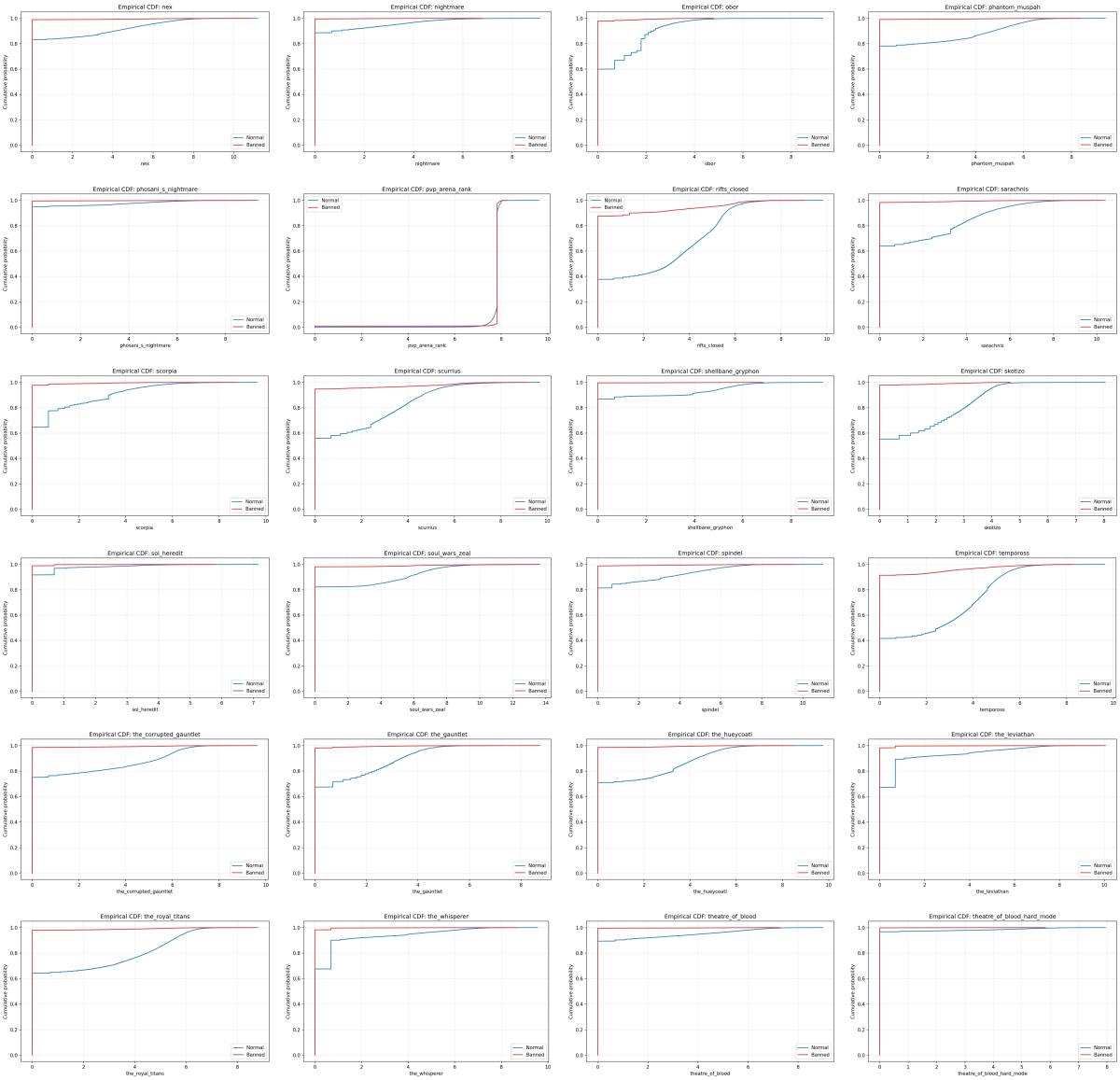

Figure 7: Empirical CDFs of raw activity counts, alphabetical (3/4).

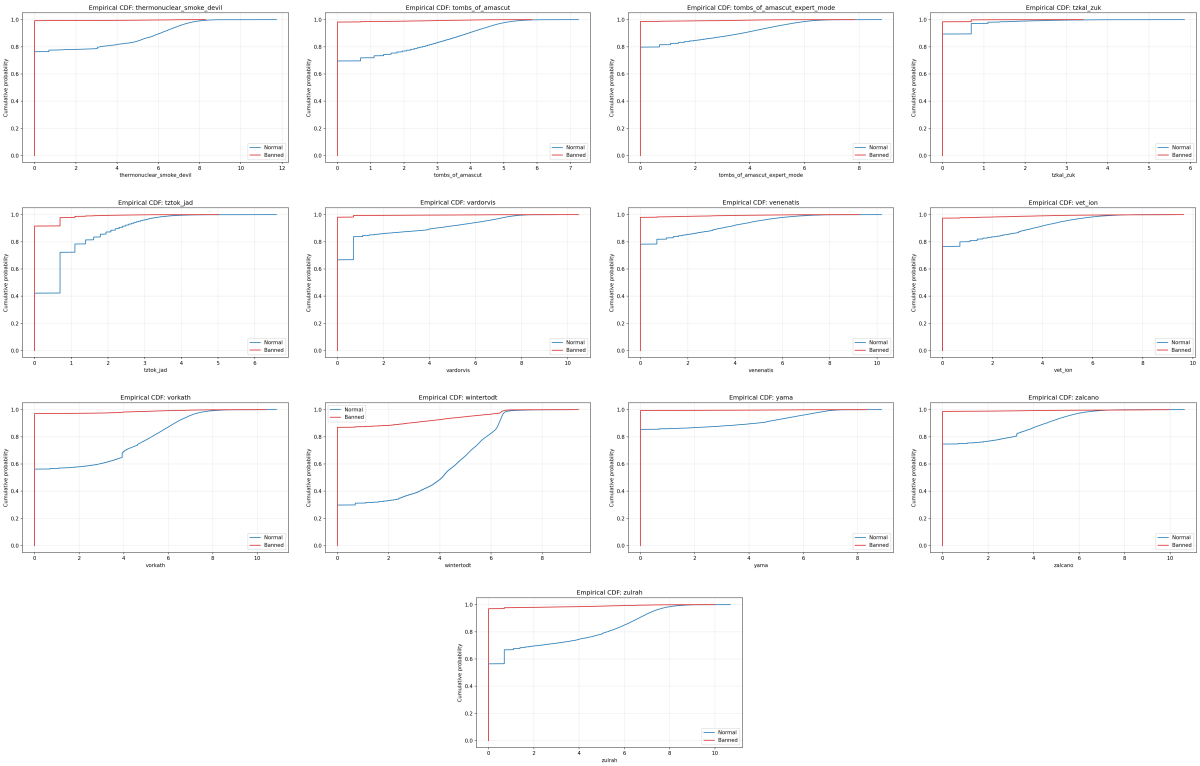

Figure 8: Empirical CDFs of raw activity counts, alphabetical (4/4).

A.3 Per snapshot velocities

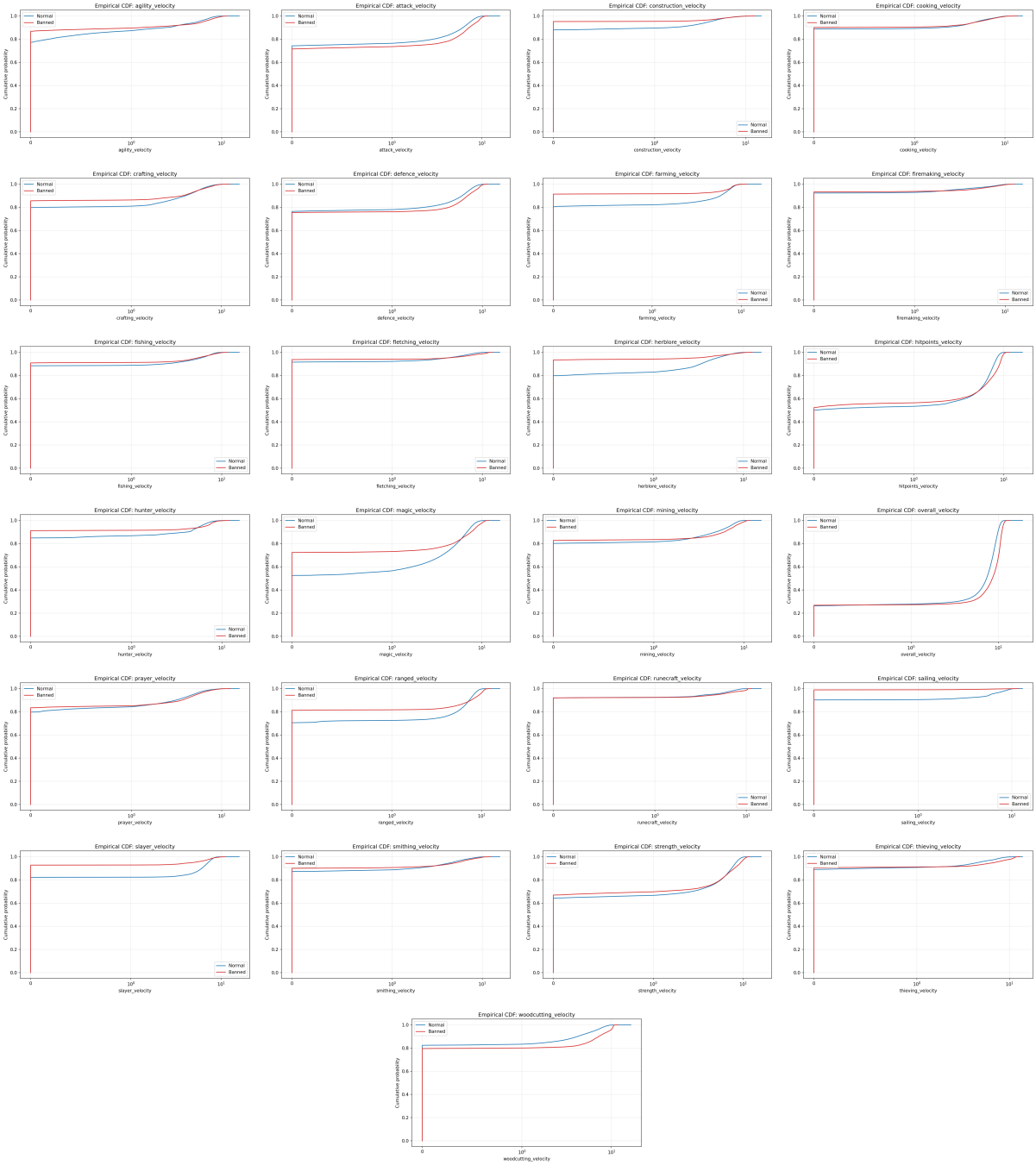

Figure 9: Empirical CDFs of per snapshot per skill XP gain velocities (XP/hr), by class.

A.4 Per skill XP ratios

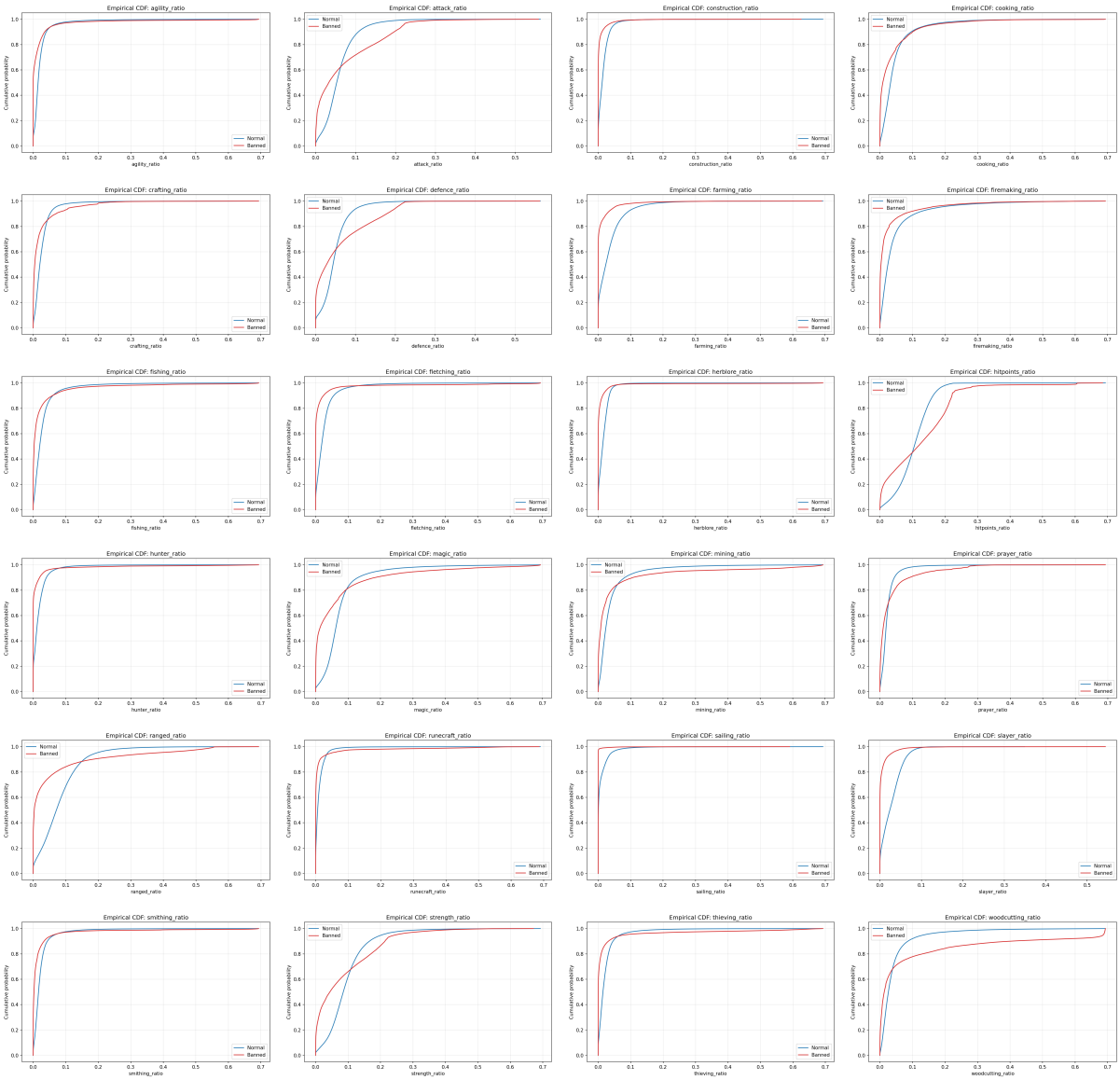

Figure 10: Empirical CDFs of per skill XP ratios $r_{i,k} = x_{i,k}/x_{i,\text{overall}}$, by class.

A.5 Derived behavioral signals

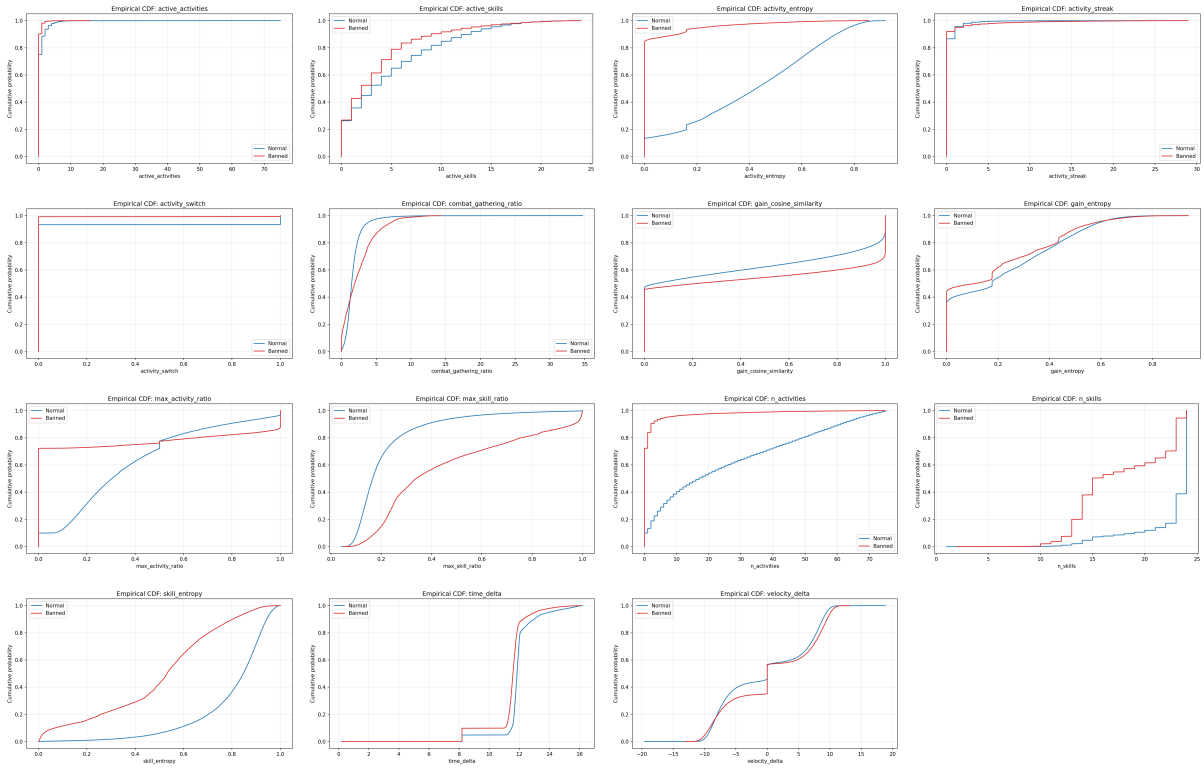

Figure 11: Empirical CDFs of derived statistics described in §6, by class. `time_delta` is the interval between snapshots and is included as a sampling diagnostic rather than a model feature.

B Additional Evaluation Statistics

Table 3: Held out validation set evaluation summary under the no horizon retained sequence protocol: each account is scored on its latest retained sequence, capped at 30 snapshots, rather than on a fixed post discovery calendar prefix. Bracketed intervals are 95% stratified bootstrap confidence intervals.

Dataset	
Total samples	169,733
Positive (banned)	35,887
Negative (normal)	133,846
Positive rate	21.14%
Core metrics	
ROC–AUC	0.9948 [0.9945, 0.9951]
PR–AUC	0.9849 [0.9841, 0.9856]
ECE (at the 10% operational base rate)	0.0020
Score distributions	
Normal mean / median / std	0.0145 / 0.0003 / 0.0764
Banned mean / median / std	0.8642 / 0.9833 / 0.2427

Table 4: Operating point performance on the held out validation set. Thresholds in the precision at fixed recall block are the calibrated score cutoffs that achieve the target recall.

Target recall	Precision	Threshold
50%	99.89%	0.983
75%	99.17%	0.859
90%	96.90%	0.494
95%	92.47%	0.219
Recall at fixed precision		
$R@P = 50\%$	99.73%	–
$R@P = 75\%$	98.90%	–
$R@P = 90\%$	96.51%	–
$R@P = 95\%$	92.99%	–

Table 5: Time horizon evaluation. Metrics are computed by restricting each account sequence to only the first d days of observed post discovery history. Bracketed intervals are 95% stratified bootstrap confidence intervals. The grid starts at $d = 4$; day 3 metrics are excluded from the headline evaluation because the day 3 cohort is an atypical subpopulation. ITAUC denotes the integrated time-AUC across $d \in \{4, \ldots, 30\}$. The $d = 30$ row is therefore distinct from the no horizon retained sequence summary in Table 3.

Days	ROC–AUC [95% CI]	PR–AUC [95% CI]	Samples	Pos. rate
4	0.9810 [0.9800, 0.9819]	0.9446 [0.9424, 0.9469]	94,974	20.70%
5	0.9817 [0.9808, 0.9825]	0.9479 [0.9457, 0.9501]	98,860	21.35%
6	0.9821 [0.9812, 0.9829]	0.9502 [0.9480, 0.9523]	101,674	21.73%
7	0.9825 [0.9817, 0.9834]	0.9518 [0.9498, 0.9539]	103,438	21.91%
8	0.9835 [0.9827, 0.9843]	0.9537 [0.9516, 0.9555]	110,110	21.50%
9	0.9839 [0.9831, 0.9846]	0.9549 [0.9529, 0.9568]	112,554	21.57%
10	0.9844 [0.9836, 0.9851]	0.9565 [0.9548, 0.9582]	115,100	21.75%
11	0.9847 [0.9840, 0.9854]	0.9574 [0.9556, 0.9592]	116,971	21.79%
12	0.9849 [0.9842, 0.9856]	0.9583 [0.9565, 0.9600]	118,970	21.89%
13	0.9851 [0.9843, 0.9858]	0.9588 [0.9570, 0.9605]	120,459	21.93%
14	0.9852 [0.9845, 0.9859]	0.9595 [0.9576, 0.9611]	121,965	22.00%
15	0.9854 [0.9847, 0.9862]	0.9601 [0.9583, 0.9618]	123,503	22.05%
16	0.9855 [0.9848, 0.9862]	0.9606 [0.9589, 0.9622]	124,244	22.14%
17	0.9856 [0.9849, 0.9862]	0.9610 [0.9591, 0.9626]	125,167	22.22%
18	0.9857 [0.9850, 0.9864]	0.9612 [0.9596, 0.9629]	126,866	22.20%
19	0.9858 [0.9851, 0.9864]	0.9615 [0.9600, 0.9631]	127,729	22.23%
20	0.9860 [0.9853, 0.9866]	0.9620 [0.9604, 0.9635]	128,501	22.28%
21	0.9860 [0.9853, 0.9866]	0.9621 [0.9604, 0.9636]	129,671	22.30%
22	0.9860 [0.9853, 0.9866]	0.9621 [0.9604, 0.9636]	130,819	22.28%
23	0.9861 [0.9854, 0.9867]	0.9622 [0.9607, 0.9637]	131,565	22.30%
24	0.9861 [0.9855, 0.9867]	0.9624 [0.9609, 0.9639]	132,213	22.34%
25	0.9863 [0.9856, 0.9869]	0.9628 [0.9612, 0.9644]	133,064	22.34%
26	0.9863 [0.9857, 0.9869]	0.9627 [0.9611, 0.9642]	134,046	22.32%
27	0.9863 [0.9857, 0.9869]	0.9627 [0.9611, 0.9642]	134,786	22.31%
28	0.9864 [0.9857, 0.9870]	0.9629 [0.9615, 0.9644]	135,394	22.31%
29	0.9865 [0.9858, 0.9871]	0.9630 [0.9615, 0.9644]	135,924	22.30%
30	0.9865 [0.9859, 0.9871]	0.9631 [0.9617, 0.9646]	136,600	22.27%
ITAUC	0.9851	0.9589	–	–